

Lecture 31 : Cryptographic hash functions (Part I)

Introduction

In the vast and interconnected landscape of digital communication, ensuring the authenticity of information and its source is paramount. Whether it's a simple email, a financial transaction, or a critical system update, users and systems need to be confident that the message received is exactly what was sent and that it originated from the claimed sender. This fundamental requirement is known as authentication, and it forms the bedrock of trust in any security application. To achieve robust authentication, one of the most vital tools in a cryptographer's arsenal is the hash function, specifically, the cryptographic hash function. This lecture delves into the foundational aspects of these functions, exploring their properties, types, and their indispensable role in securing our digital interactions. Understanding cryptographic hash functions is not merely an academic exercise; it's about grasping the underlying mechanisms that protect our data integrity and verify identities in a world where physical presence cannot always be confirmed.

Core Concepts

Term	Definition	Significance
Hash Function	A computational function that transforms an input message (M) of arbitrary length into a fixed-length output, known as a hash value or digest (H).	Essential for data processing, enabling efficient data retrieval and integrity checks in various computing contexts.
Cryptographic Hash Function	A hash function possessing specific desirable properties that make it suitable for security applications, particularly authentication and integrity verification.	Forms the backbone of digital signatures, message authentication codes, and password storage, crucial for modern cybersecurity.
One-Way Function	A function that is easy to compute in one direction (M to H) but computationally	Prevents an attacker from reconstructing the original message from its hash,

Term	Definition	Significance
	infeasible to reverse (H to M).	safeguarding data confidentiality and integrity.
Collision	An instance where two distinct input messages (x and x') produce the exact same hash value ($H(x) = H(x')$).	A fundamental challenge in hash function design; cryptographic hash functions aim to make finding collisions computationally infeasible.
Modification Detection Code (MDC)	An unkeyed hash function used primarily to ensure the integrity of a message, detecting any unauthorized alterations.	Verifies if a message has been tampered with during transmission or storage, crucial for maintaining data reliability.
Message Authentication Code (MAC)	A keyed hash function that uses a secret key (K) along with the message (M) to generate a hash value, providing both integrity and sender authentication.	Guarantees both that the message has not been altered and that it originated from a sender possessing the shared secret key.
First Preimage Resistance	The property that, given a hash value (y), it is computationally infeasible to find an input message (x) such that $H(x) = y$.	Prevents an attacker from forging a message that produces a desired hash value, protecting against message fabrication.
Second Preimage Resistance	The property that, given an input message (x), it is computationally infeasible to find a different message (x') such that $H(x) = H(x')$.	Prevents an attacker from substituting an original message with a different one that yields the same hash, crucial for non-repudiation.
Collision Resistance	The property that it is computationally infeasible to find any two distinct messages (x and x') that	The strongest resistance property, making it extremely difficult for attackers to create two messages with identical

Term	Definition	Significance
	produce the same hash value ($H(x) = H(x')$).	hashes, vital for digital signature security.

Detailed Analysis

1. Understanding Hash Functions and Their Role in Authentication

A hash function serves as a computational mechanism that takes an input message, denoted as M , which can be of arbitrary length, and transforms it into a fixed-length output, known as a hash value or hash digest, denoted as H . A key characteristic of this transformation is that the size of the input message M is typically much larger than the resulting hash digest H . This process inherently involves a many-to-one mapping, meaning multiple different input messages could potentially produce the same hash value. Due to this property, hash functions are often referred to as "one-way functions" because, while it's easy to compute H from M , it's computationally infeasible to reverse the process and uniquely determine M from H .

In the context of security applications, particularly authentication, cryptographic hash functions play a pivotal role. Authentication is the process of verifying the identity of the sender of a message. In internet communications, where direct physical verification is impossible, cryptographic hash functions provide a robust method to confirm that a message originates from the expected party. Unlike biometric authentication, which relies on physical attributes, cryptographic authentication operates on the digital representation of messages. This is crucial for a wide array of security applications, from password-based logins to secure data transfers, where confirming the sender's identity is as important as ensuring message integrity.

2. Classifying Hash Functions: Keyed vs. Unkeyed

Hash functions can be broadly categorized based on whether they utilize a secret key during their operation. This distinction leads to two primary types: unkeyed hash functions and keyed hash functions, each serving slightly different security objectives.

Unkeyed Hash Functions: Modification Detection Codes (MDC)

Unkeyed hash functions do not rely on a secret key. Their primary purpose is to ensure the integrity of a message, meaning they can detect if a message has been altered during transmission or storage. These are often referred to as Modification Detection Codes (MDCs).

The process for using an MDC is as follows:

* **Sender Side:**

- * The sender takes the original message (M).
- * A hash function (H) is applied to M to generate a hash digest, the MDC.
- * The MDC is appended to the original message M.
- * The combined message (M + MDC) is then transmitted to the receiver.

* **Receiver Side:**

- * The receiver receives the combined message (M' + MDC').
- * The receiver separates the received message (M') from the appended MDC (MDC').
- * The same hash function (H) that the sender used is applied to the received message (M') to generate a new hash digest (MDC_calculated).
- * The MDC_calculated is then compared with the received MDC'.
- * If MDC_calculated matches MDC', the message's integrity is confirmed; it has not been modified.
- * If they do not match, it indicates that the message was altered in transit, and its integrity has been compromised.

MDCs are effective for detecting accidental or malicious modifications but do not provide sender authentication, as anyone can compute the hash and append it.

Keyed Hash Functions: Message Authentication Codes (MAC)

Keyed hash functions, in contrast, incorporate a secret key (K) into the hash generation process. These are known as Message Authentication Codes (MACs) and provide a stronger form of security by ensuring both message integrity and sender authentication. The secret key K is shared only between the sender and the legitimate receiver, similar to symmetric encryption.

The process for using a MAC is as follows:

* **Sender Side:**

- * The sender takes the original message (M) and a shared secret key (K).
- * A MAC algorithm (F) is applied to M using K (F(M, K)) to generate the MAC value.
- * The MAC value is appended to the original message M.
- * The combined message (M + MAC) is then transmitted to the receiver.

* **Receiver Side:**

- * The receiver receives the combined message (M' + MAC').
- * The receiver separates the received message (M') from the appended MAC (MAC').
- * Using the same shared secret key (K) and MAC algorithm (F), the receiver computes a new MAC value (MAC_calculated) from the received message (M').
- * The MAC_calculated is then compared with the received MAC'.
- * If MAC_calculated matches MAC', both message integrity and sender authenticity are confirmed. The message has not been altered, and it must have come from someone possessing the shared secret key K.

* If they do not match, it indicates either message modification or that the message did not originate from a party with the correct secret key.

3. Desirable Properties of Cryptographic Hash Functions

For a hash function to be considered "cryptographic" and suitable for security applications, it must exhibit several crucial properties that make it resistant to various attacks. These properties ensure the one-way nature and collision resistance necessary for robust security.

- **Applicability to Arbitrary-Sized Messages:** A cryptographic hash function must be able to process input messages of virtually any length, up to a very large, practical upper limit. This flexibility allows it to be used for hashing anything from a short password to an entire file.
- **Fixed-Length Output:** Regardless of the input message's size, the hash function must always produce a hash digest of a predetermined, fixed length (e.g., 128-bit, 256-bit, 512-bit). This consistency simplifies processing and storage.
- **Computational Efficiency:** The process of computing the hash value $H(x)$ for any given message x must be computationally easy and efficient. This ensures that the hash function can be used in real-time applications without introducing significant delays.
- **First Preimage Resistance (One-Way Property):** Given a hash value y , it should be computationally infeasible to find an input message x such that $H(x) = y$. This property prevents an attacker from fabricating a message that produces a specific, desired hash value, which could be used to impersonate a legitimate message.
- **Second Preimage Resistance (Weak Collision Resistance):** Given an input message x , it should be computationally infeasible to find a different message x' (where $x' \neq x$) such that $H(x) = H(x')$. This property prevents an attacker from taking an existing message and replacing it with a different, malicious message that yields the same hash, thereby maintaining the original message's apparent integrity.
- **Collision Resistance (Strong Collision Resistance):** It should be computationally infeasible to find any two distinct messages x and x' (where $x' \neq x$) such that $H(x) = H(x')$. This is the strongest property and implies second preimage resistance. If a hash function is collision-resistant, it is extremely difficult for an attacker to create two messages that hash to the same value, which is critical for applications like digital signatures where even a single collision could undermine security.

4. Examples of Cryptographic Hash Function Families

Over the years, several families of cryptographic hash functions have been developed and widely adopted, each with different characteristics and security strengths.

- **MD (Message Digest) Family:** This family includes algorithms like MD2, MD4, and MD5. MD5, in particular, was widely used for a significant period, generating a 128-bit hash digest. However, over time, cryptographic weaknesses, specifically collision vulnerabilities, were discovered in MD5. As a result, MD5 is no longer considered secure for most cryptographic applications, though it might still be used in some niche or legacy contexts.
- **SHA (Secure Hash Algorithm) Family:** The SHA family is currently much more widely used and considered more secure than the MD family. It includes several variations that produce different hash digest sizes:
 - SHA-1 (160-bit hash) - While once popular, SHA-1 also has known theoretical weaknesses and is being phased out.
 - SHA-2 (includes SHA-256, SHA-384, SHA-512) - These algorithms produce hash digests of 256, 384, and 512 bits, respectively, and are widely used today. There are also intermediate varieties.
 - SHA-3 (Keccak) - A newer standard, selected through a public competition, offering a different design approach to hash functions.
- **RIPEMD Family:** This family includes algorithms like RIPEMD-128 and RIPEMD-160, generating 128-bit and 160-bit hash digests, respectively. They offer an alternative to the SHA family, though they are less commonly encountered in general-purpose applications compared to SHA-2.

The evolution of these hash function families highlights the ongoing arms race in cryptography, where new algorithms are developed to counter emerging threats and discovered weaknesses in older standards.

Key Takeaways

- [] Cryptographic hash functions are essential for authentication and message integrity in security applications.
- [] A hash function maps an arbitrary-length message to a fixed-length, smaller hash digest, acting as a one-way function.
- [] Authentication verifies the sender's identity, crucial in internet communications where physical presence cannot be confirmed.
- [] Unkeyed hash functions (MDCs) provide message integrity by detecting modifications during transmission.

- [] Keyed hash functions (MACs) offer both message integrity and sender authentication through the use of a shared secret key.
- [] Cryptographic hash functions must be computationally efficient and applicable to arbitrary message sizes, producing fixed-length outputs.
- [] Key security properties include First Preimage Resistance, Second Preimage Resistance, and Collision Resistance, making it infeasible to reverse the hash or find colliding messages.
- [] Popular hash function families include MD (e.g., MD5, now considered weak), SHA (e.g., SHA-256, widely used), and RIPEMD.

Conclusion

Cryptographic hash functions are the unsung heroes of digital security, silently working behind the scenes to ensure the integrity and authenticity of our online interactions. From verifying software downloads to securing communication channels and protecting sensitive data, their one-way nature and collision resistance properties make them indispensable. The distinction between unkeyed Modification Detection Codes (MDCs) for integrity and keyed Message Authentication Codes (MACs) for both integrity and authentication highlights the versatility of these tools in addressing different security requirements. As digital threats continue to evolve, the ongoing development and refinement of hash algorithms, such as the SHA family, remain critical. This foundational understanding of cryptographic hash functions sets the stage for exploring more advanced cryptographic techniques and their practical applications in securing the complex digital world we inhabit.